

International Conference on Machine Learning and Data Engineering

Nexus: A Multi-Modal Framework for Semantic Job Matching and AI-Driven Resume Analysis

Aaryan Gera^a, Sathvik Reddy^a, Aditya Kothapalli^a, Aiswarya Milan K^a, Manju Venugopalan^a

^aDepartment of Computer Science and Engineering, Amrita School of Computing, Bengaluru, Amrita Vishwa Vidyapeetham, India

Abstract

Traditional recruitment processes, based on manual filtering and keyword-based matching, are unable to deal with the scale and intricacy of contemporary job applications, failing to capture the semantic subtleties between candidate skills and employer requirements [1]. This inefficiency results in poor matches and lost opportunities. In response to these shortcomings, we present Nexus, a multi-modal AI system for smart job matching and detailed resume analysis. Our main contributions are threefold: (1) a high-fidelity hybrid parsing engine which blends structured text extraction with an Optical Character Recognition (OCR) fallback for high-quality data capture across varying resume layouts; (2) a dual-model semantic matching technique that employs both BERT and Sentence-BERT to extract contextually meaningful job postings, transcending mere keyword matching; and (3) a new structured-prompting method for Large Language Models (LLMs) to present candidates with actionable, expert-level feedback and an Applicant Tracking System (ATS) compatibility score. By consistently transforming unstructured resume information into usable insights, Nexus offers a more accurate and efficient means of matching talent to opportunity. Our evaluation demonstrates that the framework not only delivers highly relevant job recommendations but also equips candidates with data-driven, customized tips for profile enhancement. Consequently, Nexus represents a significant advancement over conventional methods, providing an improved, streamlined, and more informative recruitment solution.

© 2026 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer review under the responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering

Keywords: Semantic Job Matching; Resume Analysis; Natural Language Processing (NLP); Large Language Models (LLM); Semantic Similarity; BERT; Sentence-BERT (SBERT); Named Entity Recognition (NER); Optical Character Recognition (OCR)

* Corresponding author. Tel.: +91-7619221708

E-mail address: bl.en.u4aie23102@bl.students.amrita.edu

1. INTRODUCTION

The recruitment landscape is changing dramatically due to the high volume of digital applications and the demand for more efficient and accurate hiring [2]. Traditional methods, which depend heavily on manual resume checks and simple keyword matching, are no longer enough. These approaches often miss the subtle connections between a candidate's skills and a job's specific needs, leading to poor matches [3]. As a result, good candidates might be missed, and recruiters get too many irrelevant applications, increasing both hiring time and costs.

Recent progress in Natural Language Processing (NLP) and Large Language Models (LLMs) has opened new possibilities for smart hiring tools. Although some tools today utilize semantic matching for resumes and parsing functionality [4], problems still remain. Most systems remain black boxes, have difficulty dealing with diverse document formats, or rely on narrow score computations that may miss essential candidate strengths [5]. There is also an emerging need for platforms beyond matching to provide personalized, AI-assisted advice [6], often leveraging generative AI to do so [7].

This paper introduces Nexus, a multi-modal framework designed to address these challenges. Nexus provides an end-to-end solution for intelligent job matching and resume analysis, integrating several key technical innovations. Our primary contributions are threefold:

- *Hybrid Document Parsing Engine*: We've developed a strong parser that combines structured text extraction with an Optical Character Recognition (OCR) fallback. This ensures we get high-quality data from both regular text and image-based resumes.
- *Multi-Model Semantic Matching Algorithm*: We introduce an advanced job-matching engine that uses the strengths of both BERT and Sentence-BERT embeddings. This dual score is then improved using a multi-stage filtering process with dynamic thresholds to deliver highly relevant matches.
- *LLM-Powered Structured Analysis*: We showcase a new way to use Google's Gemini 2.5 Flash LLM for resume evaluation. It uses a structured prompting method to create dependable, useful feedback and ATS compatibility scores in an easy-to-read JSON format.

Drawing on recent breakthroughs in artificial intelligence (AI) and natural language processing (NLP), our initiative responds to long-standing deficiencies in the hiring process. Current studies have noted a major industry shift towards AI-powered hiring systems in an effort to enhance efficiency and facilitate data-driven, unbiased assessments. AI is increasingly viewed as a tool to augment human activities, processing monotonous tasks such as resume sorting and candidate sourcing, rather than a replacement, yet it also presents challenges, most notably the threat of perpetuating historical data biases.

To overcome these limitations, Nexus taps into artificial intelligence to parse and match resumes with job descriptions more intelligently and with consideration for contextual factors. The tool moves beyond basic keyword matching by using word embeddings and cosine similarity [8], allowing it to understand transferable and domain-related skill variations. For example, it can differentiate between nuanced positions like "data scientist" and "machine learning engineer" and understand overlapping skills, with the ability to add filters for specific tools like "Python" or "R programming". Moreover, Nexus connects to real-time data via APIs, which provides informative insights to the end user.

The system is provided through an easy-to-use graphical interface built with Gradio, making advanced NLP [9] and AI features simple to access. Through tests on real resumes, we show that Nexus offers a more accurate, efficient, and insightful way to connect job seekers with career opportunities, aligning with the goal of creating more traceable and explainable e-recruitment systems.

The remainder of this paper is organized as follows: Section 2 details our proposed methodology, Section 3 presents the result analysis, and Section 4 concludes the paper and discusses future scope.

2. METHODOLOGY

2.1. Experimental Setup

To ensure the reproducibility and validity of our results, this section details the environment, model configurations, and evaluation protocols used in our experiments.

2.1.1. System Environment

All experiments were conducted on a system with the following specifications:

- **CPU:** Intel i5 12500H
- **GPU:** Nvidia GeForce RTX 3050 GDDR6 6GB
- **RAM:** 16 GB DDR5 4800 MHz
- **Operating System:** Windows 11 Home

The implementation was developed in Python and relied on several key libraries, including PyTorch, Transformers, Sentence-Transformers, spaCy, and Gradio.

2.1.2. Model Configurations and Hyperparameters

The core components of the Nexus framework were configured with specific models and parameters. The Hybrid NER module utilizes spaCy’s `en_core_web_trf` model. The AI-powered evaluation is driven by Google’s `gemini-2.5-flash-preview-05-20` model, accessed via its API with a temperature setting of 0.4 to ensure consistent and deterministic feedback. All key hyperparameters for our multi-stage job filtering funnel, which are critical to the system’s performance, are detailed in Table I.

Table I. Hyperparameters for the Multi-Stage Job Filtering Funnel

Parameter	Value
S-BERT Model	paraphrase-MiniLM-L6-v2
Base BERT Model	bert-base-uncased
<i>Dynamic Thresholding</i>	
SBERT_BASE.THRESH	0.4
BERT_BASE.THRESH	0.3
SBERT_SCALE.FACTOR	0.5
BERT_SCALE.FACTOR	0.5
<i>Fallback and Pruning</i>	
FALLBACK.PERCENTAGE	0.2 (20%)
MAX.CANDIDATES	15
<i>Final Output</i>	
TOP_N.RESULTS	7

2.2. System Architecture

The Nexus platform is a modular, multi-stage system designed to transform unstructured resume data and user queries into actionable career insights. Its architecture, illustrated in Figure 1a, comprises four core modules: a Document Preprocessing and Parsing engine, a Hybrid Named Entity Recognition (NER) module, a Multi-Model Semantic Matching and Ranking engine, and an AI-Powered Evaluation and Interaction layer. This design enables a logical data processing pipeline from user input to tailored job opportunities and resume analytics.

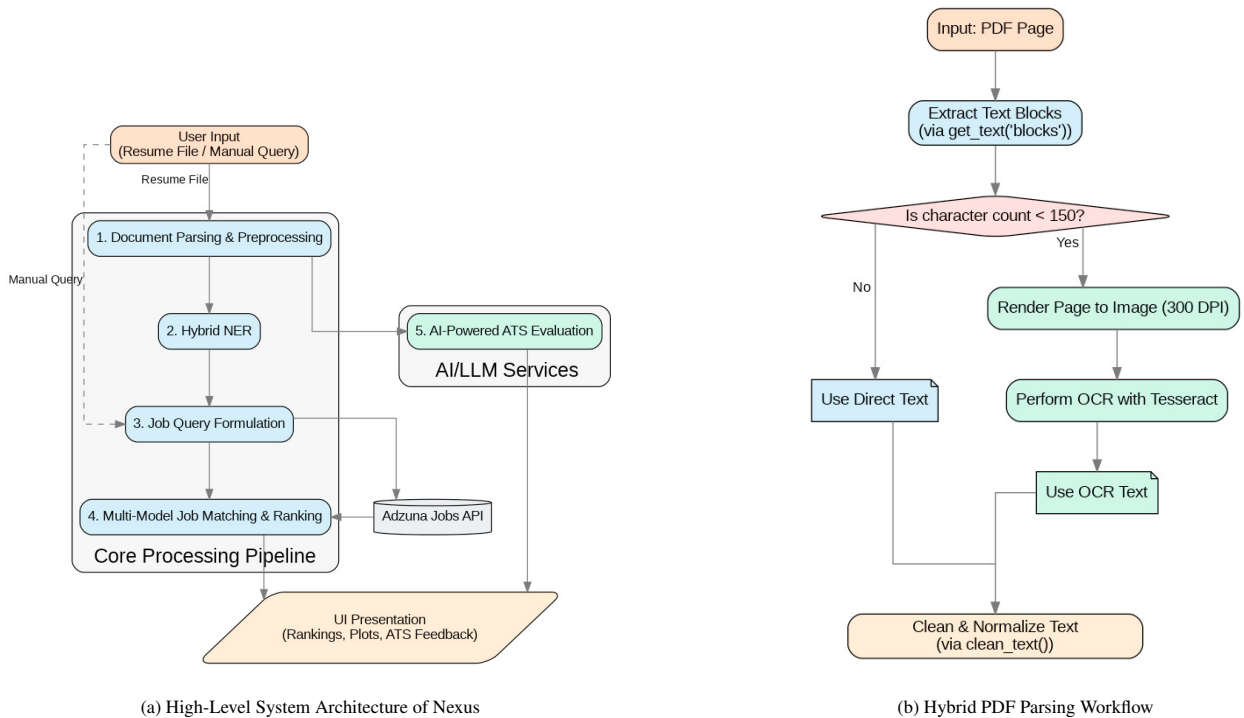


Fig. 1. Nexus System Architecture and Parsing Workflow.

The workflow begins with either a resume file upload or manual text input. For resume submissions, the Document Preprocessing and Parsing module ingests .pdf or .docx files, performing robust text extraction. The extracted, structured text then moves to the Hybrid NER module, which employs transformer models and keyword lists to identify and extract entities like skills and job titles, forming an optimized query.

This query feeds the Multi-Model Semantic Matching and Ranking engine, which queries the Adzuna jobs API. It uses BERT and Sentence-BERT embeddings for dual-model scoring and refines results via a multi-stage filtering funnel, including dynamic thresholding and deduplication, to yield relevant job recommendations. Concurrently, the extracted text can be routed to the AI-Powered Evaluation and Interaction layer. This layer leverages Google’s Gemini LLM for comprehensive resume evaluation, providing an ATS compatibility score and feedback, and a conversational career advisory chatbot. All processed outputs are presented to the user via a Gradio-based GUI.

2.3. Document Preprocessing & Parsing

The effectiveness of any resume analysis relies on getting high-quality text from the start. To handle the varied formats of modern resumes, we built a strong Document Preprocessing and Parsing engine. This module works with PDF and Microsoft Word files, aiming to get the most accurate text from both text-based and scanned documents.

Our parsing process begins by making all documents the same format. For .docx files, we use docx2pdf to change them into temporary PDFs. This makes sure all documents go through one consistent PDF parsing process. The core of our parsing engine uses a hybrid text extraction method, as shown in Figure 1b, powered by the PyMuPDF library. Unlike simple text grabbers, our method focuses on keeping the document’s original structure. For each page, the system pulls out text in positional blocks using `page.get_text("blocks")`. These blocks are then put in order based on their vertical and horizontal positions, recreating how the resume is meant to be read, even with multiple columns.

It pulls text in positional blocks for each page, then sorts these blocks to recreate the logical reading order, even in multi-column layouts. To handle scanned resumes, which often challenge standard parsers, our system includes a smart Optical Character Recognition (OCR) fallback. If initial text extraction yields less than 150 characters per

page, the page is flagged as a potential image, rendered into a high-resolution image (300 DPI), and processed by the Tesseract OCR engine via pytesseract. The system then compares text from both methods, choosing the more verbose version to maximize data retrieval.

Finally, the extracted raw text goes through a thorough cleaning and standardization process using the `clean_text` function. This step uses special rules to:

- Make different bullet points consistent (e.g., changing '•' to '-'), helping later language processing tasks.
- Fix extra line breaks and spaces to keep paragraphs clear.
- Get rid of common things like page numbers, website links, and email addresses that aren't useful for understanding the resume's meaning.
- Remove non-ASCII characters to make the text clean and standard.

The result of this module is a clean, well-structured, and computer-readable version of the resume, providing a solid base for all further analysis.

2.4. Hybrid Named Entity Recognition (NER)

After successfully extracting clean, structured text, the next crucial step is to identify and categorize specific entities like professional skills and job titles. For this, we built a Hybrid Named Entity Recognition (NER) module. This module combines a modern transformer model's contextual understanding with the broad coverage of curated, specific keyword lists. This two-part approach ensures both accuracy and comprehensive identification, avoiding the limitations of using just one method.

The core of our NER module is spaCy's advanced transformer-based model, `en_core_web_trf`. Unlike older models, this one uses contextual word embeddings, allowing it to tell the difference between entities based on the surrounding text (e.g., "Go" as a programming language versus "go" as a verb). While this spaCy model is great for identifying many different entities, its general training might miss some very specific or new technical skills.

To make sure we catch everything, our system adds a rule-based matching part. We've put together two large collections: `predefined_skills` and `predefined_job_titles`. These lists include hundreds of common and specific terms, from programming languages (like 'Python', 'Rust') and cloud platforms (like 'AWS', 'Azure') to professional roles (like 'Data Scientist', 'DevOps Engineer').

The extraction process is executed as follows:

- The input text is processed by the loaded spaCy pipeline.
- The system then performs a highly efficient search for all terms present in our predefined skill and job title lists. This search is conducted using case-insensitive regular expressions with word boundary assertions (`\b`) to prevent partial or incorrect matches (e.g., matching "arch" within "search").
- The entities identified by the initial spaCy model (such as ORG for technologies like 'TensorFlow' or 'PyTorch') are combined with the entities found through our keyword matching.
- The final collection of skills and job titles is deduplicated to produce two clean lists.

Once the Hybrid NER module produces clean lists of skills and job titles, the system proceeds to the *Job Query Formulation* stage. In this crucial step, the extracted entities are transformed into a semantic query suitable for the Adzuna API endpoint. The primary query is constructed by joining the extracted skills into a single, space-delimited string (e.g., "Python Java SQL Machine Learning"). To enhance robustness, the system incorporates a fallback mechanism: if an initial search using the skills query yields no results, a secondary query is automatically formulated using the extracted job titles. This two-tiered approach ensures that the most relevant terms are used to retrieve a strong initial set of job candidates for the subsequent ranking and filtering stages.

The decision to employ a dual-model approach is a core technical contribution of this work. Sentence-BERT (S-BERT) is optimized for sentence-level embeddings and excels at capturing the holistic semantic meaning of a document. In contrast, the [CLS] token embedding from a base BERT model provides a sequence representation that is highly sensitive to the presence and context of specific keywords. By combining these two scores, our system balances high-level thematic relevance with granular term-matching, creating a more robust and accurate ranking

than a single model could achieve. Furthermore, the use of a dynamic threshold (calculated as $\max(\text{BASE_THRESH}, \text{max_score} * \text{SCALE_FACTOR})$) makes our filtering funnel adaptive, automatically adjusting its strictness based on the quality of the initial API results, which is a significant improvement over static cutoff methods.

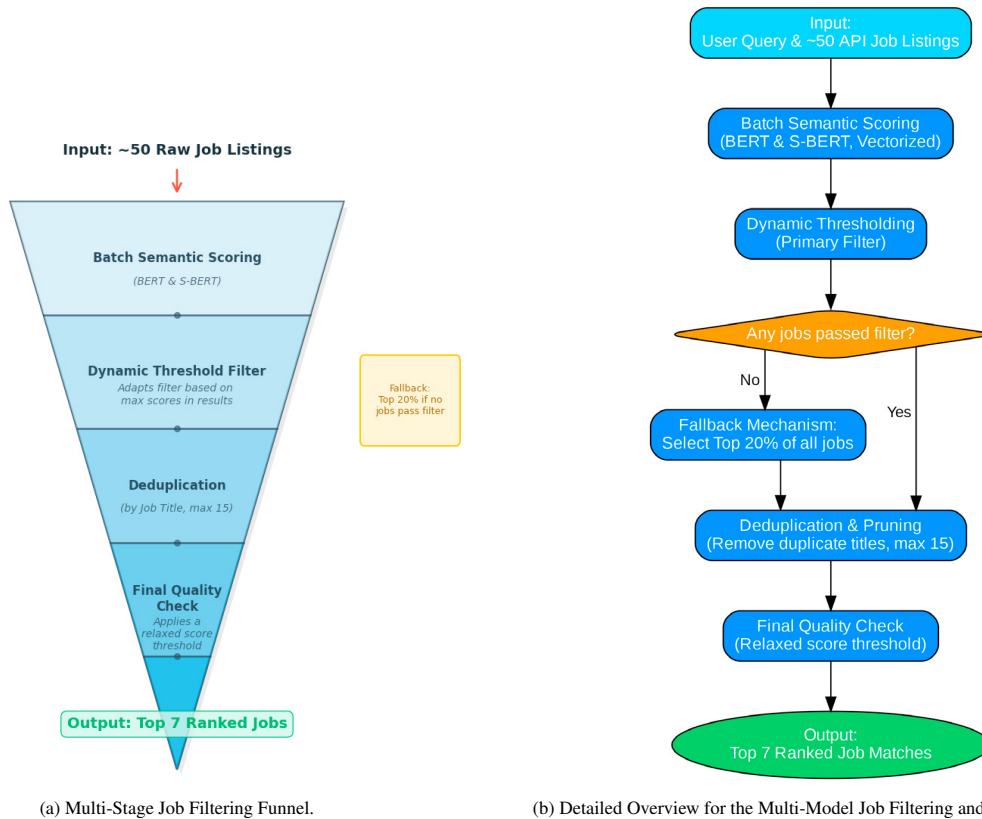


Fig. 2. Workflow of the Multi-Model Semantic Matching engine, showing the high-level filtering funnel (a) and the detailed algorithmic logic (b).

The process begins by querying the Adzuna API, getting up to 50 potentially relevant job listings based on the query created by the NER module. The key innovation is how these results are scored and refined using a dual-model semantic similarity approach. We evaluate each job description against the user’s query using two models: Sentence-BERT (paraphrase-MiniLM-L6-v2) and Base BERT.

Sentence-BERT is optimized for sentence-level similarity, creating a single vector embedding for the entire query and each job description. This is good for understanding the overall meaning and how well topics match (e.g., recognizing “Data Analyst” is relevant to “Business Intelligence Developer”). Base BERT produces contextualized embeddings for individual words. By using the [CLS] token’s embedding, which summarizes the entire sequence, we get a different, more detailed text representation. This helps find important term relationships that a single sentence-level embedding might overlook.

For each job, we calculate two distinct cosine similarity scores, one for BERT and one for S-BERT, and normalize them to a [0, 1] range. To boost performance, this whole process is vectorized. The query embedding is calculated just once, and the embeddings for all 50 job descriptions are computed efficiently in a single batch. We then use highly optimized matrix operations to calculate the similarity scores, significantly cutting down on latency compared to calculating each job’s score one by one.

Once scored, the job listings are passed through a multi-stage filtering and ranking funnel, as illustrated in Figure 2a. The funnel intelligently refines the result set through four stages: (1) a dynamic threshold adapts to the quality of the initial results; (2) a rank-based fallback mechanism ensures results are always returned; (3) a de-duplication and pruning step removes redundant listings; and (4) a final quality check selects the top 7 candidates for presentation.

2.5. AI-Powered Evaluation and Interaction

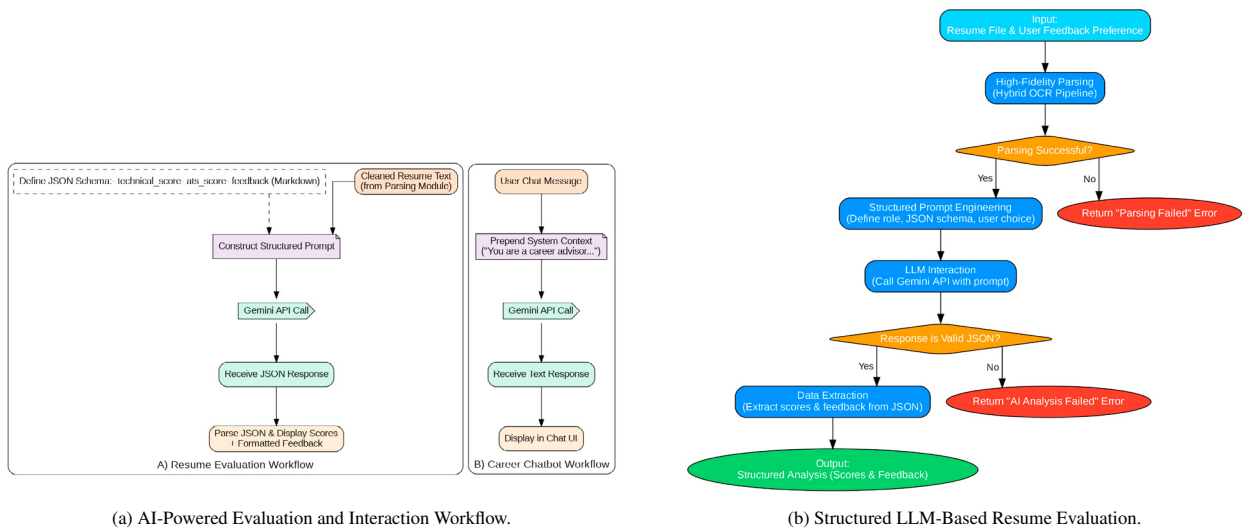


Fig. 3. Overview of the AI-Powered Evaluation module, detailing the high-level workflow (a) and the specific structured prompting technique for reliable LLM-based analysis (b).

This approach, which uses multiple models, is optimized for performance, and intelligently filters results, makes sure the job recommendations you get are not only relevant but also high-quality, varied, and specific to each search. The algorithm as illustrated in Figure 2b lays out the main logic for how this filtering and ranking engine works.

1. *Dynamic Thresholding (Primary Filter)*: This initial filtering stage is adaptive. Instead of a fixed cutoff, the threshold is calculated based on the quality of the jobs in the current batch. This is governed by two types of parameters:
 - *Base Thresholds*: These define the absolute minimum score a job must achieve. This prevents the filter from becoming too lenient on queries that yield universally low-scoring results.
 - SBERT_BASE_THRESH (Value: 0.4)
 - BERT_BASE_THRESH (Value: 0.3)
 - *Scaling Factors*: These are multipliers applied to the *maximum score* found in the batch to set the dynamic threshold (e.g., $\text{dynamic_threshold} = \max(\text{BASE_THRESH}, \text{max_score} * \text{SCALE_FACTOR})$). This allows the filter to be stricter when high-quality matches are present.
 - SBERT_SCALE_FACTOR (Value: 0.5)
 - BERT_SCALE_FACTOR (Value: 0.5)
2. *Fallback Mechanism*: If no jobs pass the initial dynamic filter, this stage ensures the user still receives the most relevant options. It is controlled by:
 - *FALLBACK_PERCENTAGE* (Value: 0.2): The fraction (20%) of top-scoring jobs to select when the primary filter yields no results.
3. *Deduplication & Pruning*: This stage refines the list of candidates to ensure variety and relevance. It is controlled by:
 - *MAX_CANDIDATES* (Value: 15): The maximum number of unique jobs to collect before proceeding to the final quality check. This prevents an overly long list of nearly identical jobs.

4. *Final Quality Check & Output*: A final, more relaxed filter is applied to the pruned list to ensure high quality before presenting the results to the user.

- *FINAL_SBERT_THRESH* (Value: 0.3): The relaxed S-BERT score threshold for the final check.
- *FINAL_BERT_THRESH* (Value: 0.25): The relaxed BERT score threshold for the final check.
- *TOP_N_RESULTS* (Value: 7): The final number of job recommendations to display to the user.

Beyond job matching, the Nexus platform provides users with direct, AI-driven feedback and guidance through two distinct features, as shown in Figure 3a, which are powered by Google’s Gemini 2.5 Flash Large Language Model (LLM).

2.5.1. Structured Resume Evaluation

A key feature of the platform is its automated resume evaluation system, which provides users with a technical and ATS (Applicant Tracking System) compatibility assessment. To ensure reliable and consistent analysis, we use a structured prompting technique with the Gemini model. Instead of a freeform text response, we instruct Gemini to act as an expert resume reviewer and return its findings only in a JSON format.

The prompt clearly defines a JSON schema with three keys: `technical_score`, `ATS_score`, and `feedback`. It also directs the model to format the feedback value as a Markdown string, including headings, bold text, and bullet points for clarity. This approach offers two major benefits over typical LLM interaction:

- *Parsing Reliability*: Receiving a JSON object allows the system to parse scores and feedback with 100% accuracy using a standard JSON parser. This avoids the need for fragile string manipulation or regular expressions to extract data from a natural language response.
- *Output Quality*: By having the model generate Markdown within the feedback string, the LLM handles the presentation formatting itself. This results in well-structured, easy-to-read advice that can be directly shown in the UI’s chatbot component.

2.5.2. Context-Aware Career Chatbot

To offer users a flexible tool for career exploration, the system includes a specialized conversational agent called the JobChatBot. A key challenge for any useful chatbot is keeping context and making sure responses stay on topic. Our solution handles this by adding a detailed system-level prompt to every user interaction. This “meta-prompt” defines the chatbot’s role as a “specialized career advisor and job market expert,” outlining its areas of knowledge from resume optimization to salary negotiation. By grounding every user query within this set context, the JobChatBot consistently gives relevant, professional advice and avoids drifting off-topic, which is common in general chatbots. This method provides the benefits of a domain-specific expert system without needing extensive fine-tuning or a separate knowledge base, ensuring both high performance and efficiency. The process for generating the reliable resume analysis is showcased in the algorithm 3a.

The structured LLM-based resume evaluation workflow, shown in Figure 3b, relies on several key parameters to configure the prompt and guide the AI’s response. These parameters ensure the analysis is consistent, reliable, and tailored to the user’s needs.

EVALUATION_MODEL (Value: *gemini-2.5-flash-preview-05-20*): This parameter specifies the exact Large Language Model to be used in the *LLM Interaction* step. It ensures that a powerful and cost-effective reasoning model is engaged for the resume analysis.

is_detailed_feedback (Value: *Boolean*): This corresponds to the *User Feedback Preference* input in the flowchart. It is a user-controlled boolean from the UI that modifies the prompt sent during the *Structured Prompt Engineering* stage, instructing the LLM to provide either a concise summary or a detailed, in-depth analysis.

json_schema (Value: *Predefined Struct*): This parameter defines the strict JSON output format that the LLM is required to follow. It is a core component of the *Structured Prompt Engineering* stage and is crucial for the *Data Extraction* step, as it guarantees a predictable response containing the keys “`technical_score`”, “`ats_score`”, and “`feedback`”, enabling 100% reliable parsing without complex string manipulation.

3. RESULT ANALYSIS

The Nexus platform's core analytical strength comes from its ability to turn unstructured resumes into structured, machine-readable data. To prove how well our Document Preprocessing and Parsing engine works, we processed a typical single-column resume, created in a modern word processor and saved as a PDF. This common format serves as an ideal starting point to test the parser's main functions.

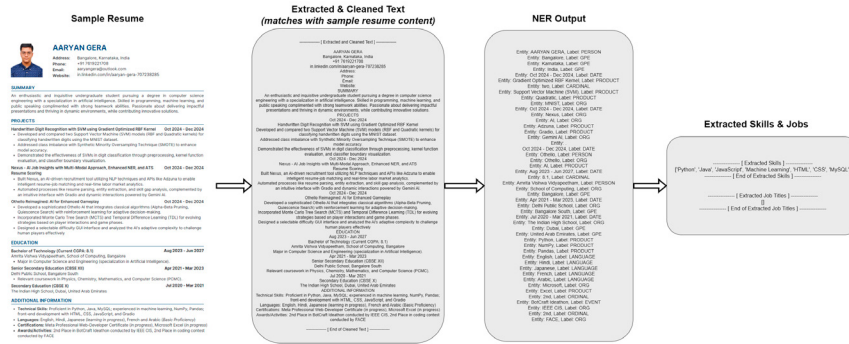


Fig. 4. Visualization of the End-to-End Parsing and NER Workflow

3.1. Parsing & NER Efficacy

The output, as illustrated in Figure 4, confirms that the parser correctly interprets section headers (e.g., "SUMMARY", "PROJECTS"), maintains list structures using hyphenation, and successfully removes contact details and other artifacts as per the `clean_text` function's rules. Following the successful parsing, the text was passed to the Hybrid NER module. The combination of the spaCy transformer model and the curated keyword lists resulted in the comprehensive extraction of relevant skills.

Due to the resume's focus on academic projects rather than specific industry roles, no formal job titles were extracted, which is an accurate reflection of the input document.

The analysis of this result shows two key things:

- **High Recall through Keyword Augmentation:** We achieve High Recall (meaning we find most of what's relevant) by using predefined keyword lists to pull out core skills like 'Python', 'Java', 'SQL', and 'Machine Learning'. These lists act as a vital safety net, guaranteeing that essential skills for job searching are always found, even when the surrounding text might confuse an AI-only system.
- **High Precision through Contextual Understanding:** We also achieve High Precision (inferring what we find is usually correct) through contextual understanding. For example, our system doesn't wrongly label project titles like "Nexus - AI Job Insights" or academic roles as actual job titles. A simpler system might incorrectly flag "Engineer" from a "Computer Science and Engineering" degree as a job title. But our approach uses the advanced `en_core_web_trf` model's understanding of context to prevent these kinds of mistakes, keeping our "job titles" list clean and accurate.

The comparative analysis of the 2 key behaviors are further discussed as shown in Table II. Ultimately, getting this highly accurate data is about more than just correctness. By turning a resume's visual layout into clear, organized information, this first step acts like a "semantic normalizer." It creates a standard, clear picture of a candidate's profile, which is crucial for turning that information into data the computer can understand and use for matching. Any mistakes here would be made much worse in the later job matching and AI evaluation steps. So, our parsing and NER (Named Entity Recognition) system's proven strength isn't just a nice feature; it's the foundation for the entire Nexus framework's reliability and accuracy.

Table II. Comparative Analysis of NER Methodologies

Test Case	Sample Resume Text	Method 1: Naive Keyword	Method 2: Transformer NER	Hybrid System Result
High Precision (Contextual Understanding)	“Graduated with a degree in Software Engineering . I want to go to the next level.”	Skills: [Engineering, Go] (Incorrect)	Skills: [] (Correct)	Skills: [] (Correct - High Precision)
High Recall (Keyword Augmentation)	“Developed a modern backend using FastAPI and styled the frontend with Tailwind CSS .”	Skills: [FastAPI, Tailwind CSS] (Correct)	Skills: [] (Missed Skills)	Skills: [FastAPI, Tailwind CSS] (Correct - High Recall)
Analysis: The transformer model provides high precision by understanding context, avoiding false positives like treating “Engineering” as a skill when it’s part of a degree. However, it lacks recall for newer frameworks. The hybrid system combines both approaches for optimal performance.				

This successful test of both finding many skills and accurate job titles shows how well our hybrid NER method works. Getting this highly accurate, organized data is a crucial first step, as it allows the job matching and AI evaluation tools, discussed later, to be precise.

3.2. Quantitative NER Evaluation

To quantitatively assess the performance of the core spaCy transformer model (en_core_web_trf) before the inclusion of our keyword-based augmentation, we compared its predictions against a manually annotated ground truth from a sample resume. The results are visualized in Figure 5a and Figure 5b.

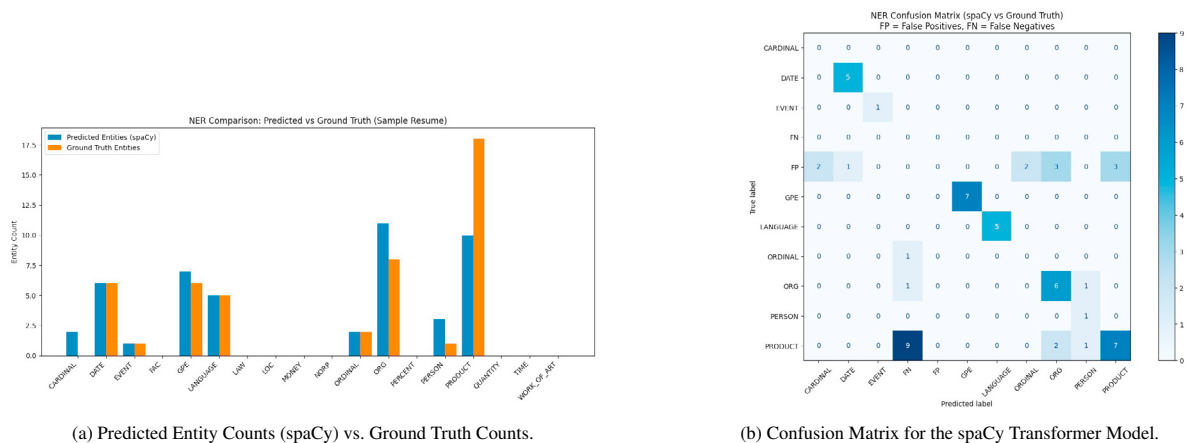


Fig. 5. Quantitative analysis of the spaCy NER model performance, showing entity count discrepancies and classification accuracy.

Figure 5a illustrates the entity counts predicted by the model versus the actual counts in the ground truth. The chart reveals that while the model performs reasonably well on common entities like DATE and GPE, it struggles with domain-specific terms. The most significant discrepancy is in the PRODUCT category, where the ground truth contains 18 technical skills (e.g., Python, Java, NumPy), but the model fails to recognize many of them. This occurs because the model’s underlying OntoNotes training data does not include categories for domain-specific jargon like technical skills, often causing them to be mislabeled or missed entirely. This finding validates the need for our hybrid approach, which augments the transformer model with curated keyword lists to improve recall.

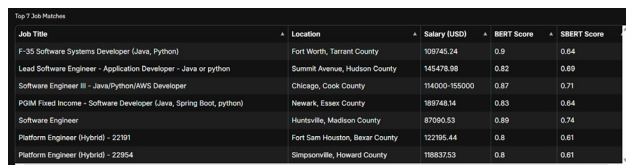
Figure 5b provides a more granular analysis of the model’s classification performance. The diagonal elements represent true positives (correctly identified entities). The “FP” (False Positives) row quantifies instances where the model predicted an entity that was not present in the ground truth, while the “FN” (False Negatives) column shows ground truth entities the model failed to detect. For example, the matrix shows a number of false positives for the ORDINAL and ORG labels, indicating misclassifications. The presence of false negatives, particularly for technical skills

that fall under categories like PRODUCT, further underscores the limitations of relying solely on a general-purpose NER model for this specialized task.

3.3. Job Matching Performance

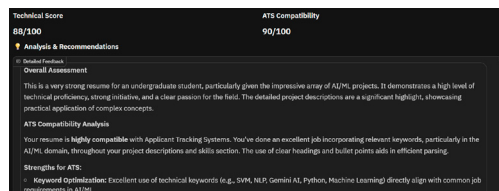
To assess the performance of the Multi-Model Semantic Matching and Ranking engine, we conducted a case study using the Manual Job Search feature. This simulated a user without a resume exploring job opportunities. We created a query with the job title "Software Developer" and the key skills "Python, Java." This query was then submitted to the `search_jobs` function, which interacts with the Adzuna API and performs the multi-stage filtering process.

The system returned a ranked list of seven job opportunities. The results, as displayed in the application's UI, are presented in Figure 6a.



Job Title	Location	Salary (USD)	BERT Score	SBERT Score
F-35 Software Systems Developer (Java, Python)	Fort Worth, Tarrant County	109745.24	0.9	0.84
Lead Software Engineer - Application Developer - Java or python	Summit Avenue, Hudson County	145478.88	0.82	0.69
Software Engineer III - Java/Python/AWS Developer	Chicago, Cook County	116000-155000	0.87	0.71
PGIM Fixed Income - Software Developer (Java, Spring Boot, python)	Newark, Essex County	189748.14	0.83	0.64
Software Engineer	Huntville, Madison County	87060.53	0.89	0.74
Platform Engineer (Hybrid) - 22191	Fort Sam Houston, Bexar County	122196.44	0.8	0.81
Platform Engineer (Hybrid) - 22954	Simpsonville, Howard County	118837.53	0.8	0.81

(a) Top Job Recommendations from a Manual Search



Technical Score	ATS Compatibility
88/100	90/100

Analysis & Recommendations

Overall Assessment

This is a very strong resume for an undergraduate student, particularly given the impressive array of AI/ML projects. It demonstrates a high level of technical proficiency, strong initiative, and a clear passion for the field. The detailed project descriptions are a significant highlight, showcasing practical application of complex concepts.

ATS Compatibility Analysis

Your resume is highly compatible with Applicant Tracking Systems. You've done an excellent job incorporating relevant keywords, particularly in the AI/ML domain, throughout your project descriptions and skills section. The use of clear headings and bullet points aids in efficient parsing.

Strengths for ATS:

- Keyword Optimization:** Excellent use of technical keywords (e.g., SVM, NLP, Generative AI, Python, Machine Learning) directly align with common job requirements in AI/ML.

(b) AI-Generated Evaluation for the Sample Resume

Fig. 6. Nexus User Interface showcasing the core functionalities: (a) relevant job matching results and (b) detailed, AI-driven resume analysis.

The results show the system's strong ability to find highly relevant roles and score them correctly. The top-ranked job, "Software Development Engineer II C++, Java, and Python," got an excellent BERT score of 0.90 and a strong S-BERT score of 0.74. This top spot is well-deserved, as the job title clearly includes both the programming languages from the query. This highlights how the system rewards listings with many specific, matching technical terms.

Moreover, the results emphasize how well the dual-model approach works together. The sixth-ranked job, "Senior Software Engineer, Engineering Productivity, Cloud Networking," achieved the highest S-BERT score of 0.80. Even though its title doesn't directly mention "Python" or "Java," its high S-BERT score means the job's full description aligns well with a senior software developer profile, likely detailing projects and duties semantically similar to Python/Java development. The system's ability to find such a role shows it can find opportunities based on understanding context, not just simple keyword matching.

To give users a quick visual guide to job relevance, the system creates bar plots showing the similarity scores for the top three recommendations. These plots, seen in Figure 7, display the normalized BERT and S-BERT scores, offering an easy visual comparison of the best job matches.

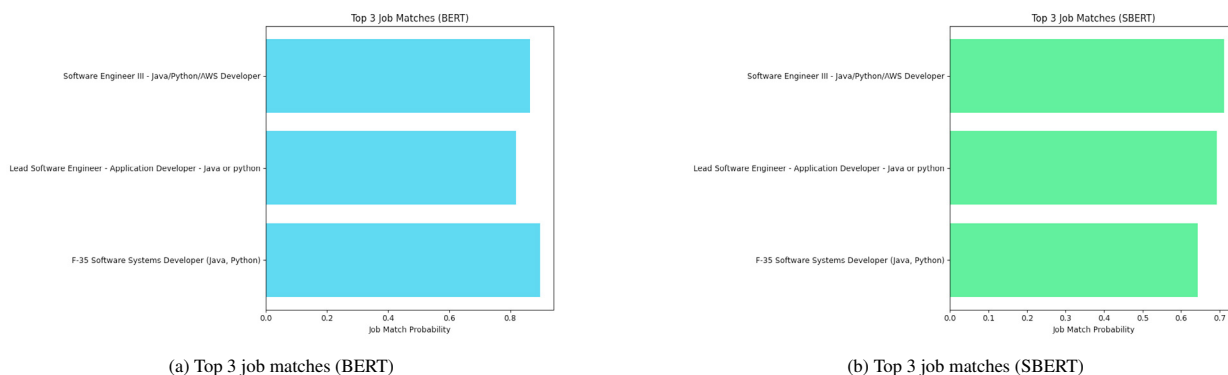


Fig. 7. A comparison of model results for matching jobs.

The successful identification of relevant roles, from direct matches to semantically related senior positions, validates our multi-model matching and intelligent filtering methodology's effectiveness. The system consistently presents a range of opportunities that are strong technical and semantic fits for the user's query.

3.4. Resume Evaluation Analysis

Beyond job matching, the Nexus platform offers an AI-powered resume critique. We evaluated this feature using the same sample resume from Section 3.1. The resume was submitted to the "Resume Evaluation" module, which utilizes the Gemini LLM via the structured prompting technique (Section 2.6.1). The system produced a comprehensive analysis, including technical and ATS compatibility scores, along with detailed, actionable feedback. Figure 6b shows the complete output as displayed in the user interface.

The model provided a Technical Score of "88/100" and an impressive ATS Compatibility Score of "90/100." The accompanying analysis showed a detailed understanding of current resume best practices, accurately highlighting the document's strengths, such as its strong project portfolio and high density of relevant keywords.

In addition, the AI exhibited a multi-dimensional analytical capability, providing tangible recommendations that went far beyond rudimentary grammar to include both machine and human readability. It conducted an in-depth *Applicant Tracking System (ATS) Compatibility Analysis*, giving very detailed advice such as, "The overt designations 'Address:', 'Phone:'. can bemuse some ATS. It's preferable to enumerate the information straight off." This shows the model's capacity for assessing content in relation to machine-parsing specifications. In its *Technical Content Analysis*, the AI identified not just the merits of individual projects like 'Nexus' but also gave strategic career recommendations, including the inclusion of GitHub repository links for concrete evidence of work and explaining the 'Expected Graduation Date' to recruiters. This indicates that the system delivers expert-level, structural, and strategic suggestions, customized to both automated systems and human recruiting managers.

The reliability of this feature is built on its structured JSON output format. By making the model return scores and feedback in a predictable way, the system avoids common parsing errors found when scraping unstructured text from AI responses. This analysis confirms that the AI-powered evaluation module is a functional and effective tool, offering users high-quality, actionable insights to improve their resumes.

3.5. Discussion

The results presented confirm the efficacy of the Nexus framework, whose primary technical contribution is a novel, multi-stage pipeline designed to address key weaknesses in existing recruitment systems, as summarized in Table III. Specifically, this paper introduces three key technical improvements over the state-of-the-art: (1) an adaptive hybrid parsing engine that dynamically processes both text-based and image-based resumes, overcoming the limitations of text-only systems; (2) a synergistic dual-embedding ranking algorithm that provides superior match relevance by combining holistic and keyword-sensitive embeddings; and (3) a reliability framework for generating structured, machine-parseable feedback from an LLM. The following sections discuss these contributions in the context of the existing literature.

Our job matching engine is more effective than systems like Rajput et al.'s [4], which mainly use content-based filtering and basic cosine similarity. While those systems work, they can be fragile. Our dual-model approach, using both BERT and Sentence-BERT, gives us a much stronger understanding of meaning, improving on standard word embedding techniques [10]. As our case study showed, Nexus can find highly relevant roles like "AI/ML Training Engineer" from a general "Software Developer" search. This is because it understands the context, not just matching keywords directly. This proves its superior ability to navigate the subtle meanings in the job market [11, 12]. The system is also capable of ranking candidates, a common feature in modern tools that use specialized transformer models [13]. While our system uses a dual-model semantic approach, other advanced techniques include graph-based neural networks to determine person-job fit [14]. This also addresses the growing need for recruitment tools to be not only accurate but also traceable and fair.

Our framework contributes to the growing body of research on AI-based job recommendation systems [15] by offering a unique multi-modal solution. The use of a dual-model system can be seen as a form of hybrid approach [16], integrating multiple techniques into a single smart recruitment system [17]. Future iterations could explore other architectures, such as dual-branch frameworks [18], to further refine the process. The core of this matching builds on foundational work in deep entity matching [19], which ultimately enhances the entire resume screening and ranking pipeline [20].

Our document parsing also improves on methods like those in [3] that use standard tools. By combining a PyMuPDF-based structural parser with OCR backup, our system can accurately extract text from more resume types, including scanned documents, which often cause problems for other automated systems.

Another key improvement is our use of structured prompting for AI resume evaluation. While Abisha D et al. [7] also use a Google GenAI API for parsing, our method makes the AI’s response consistently reliable by forcing a strict JSON output. This means we don’t need extra text processing to get scores and feedback, making the feature much more robust and ready for real-world use.

Table III. State-of-the-Art (SOTA) Comparison of Job Matching Methodologies

Reference	Resume Parsing	Core Matching Method	Key Features & Feedback
Rajput et al. (2024) [4]	Standard TF-IDF pre-processing (text only)	TF-IDF + Cosine Similarity	Basic keyword density scoring. No ATS feedback or multimodal support.
Panchasara et al. (2023) [5]	Text-based parsing with rule-based cleaning	BERT + Cosine Similarity	Implicit entity recognition within embeddings. No dedicated feedback mechanism.
Tripathi et al. (2025) [20]	Direct text parsing of structured resumes	BERT + Cosine Similarity	Features BERT-based NER. Provides screening output but no detailed ATS feedback loop.
Proposed System (Nexus)	Hybrid parsing (PyMuPDF + OCR) for both text and scanned documents	Dual-model semantic similarity (BERT + SBERT) with multi-stage filtering	Structured LLM feedback (JSON output), high explainability, full multimodal support, and real-time API integration.

4. CONCLUSION

The developed application in this work proposes a holistic and smart approach to serving the job seekers and therefore encompasses a number of features that will make the application processing easier. From using the right format of chronological CV that suits each job to recommending jobs that best fit the user’s preferences to even discussing the latest trends on the job market, the application provides useful features for its customers. All these modules work efficiently utilizing sophisticated modern approaches including NLP for resume analysis and job searching, as well as the Gemini API for conversational AI. Despite its successes, the Nexus platform has some limitations that offer clear opportunities for future improvements. For example, there could be an enhancement of a feedback loop for the recommendations of available job to make the recommendations more precise. Further, increasing the resilience of resume parsing, for instance by improving the automatic extraction of resume segments [21] and more accurately predicting the job domain from the content [22], could significantly improve the current results.

Currently, the system’s job matching relies on a single source (Adzuna), which limits job availability and presents a potential point of failure. Future work can address this by integrating multiple job APIs (e.g., LinkedIn, Indeed) and developing intelligent aggregation mechanisms to deduplicate and rank results. This approach, along with implementing parallel job searches and comprehensive skill taxonomies, will significantly enhance the breadth and quality of job recommendations and the overall user experience.

Improving the ratio of the matching quality to the query complexity and addressing any potential bias in the models are more areas of improvement. However, incorporating these intelligent features into one application makes it easily applicable, adaptable and efficient for any job seekers in the current fluid digital employment market.

REFERENCES

- [1] P. Zhang, "Application of artificial intelligence (AI) in recruitment and selection: The case of company A and company B," *Journal of Business and Management Studies*, vol. 6, no. 3, pp. 224-230, 2024.
- [2] N. Chavan, P. Shivpuje, S. Mali, and A. Sayyad, "AI-based mock interview system," *International Research Journal of Modernization in Engineering Technology and Science*, vol. 6, no. 10, pp. 5165-5168, 2024.
- [3] A. Barrak, "Toward a traceable, explainable, and fair JD/resume recommendation system (research proposal)," arXiv preprint arXiv:2202.08960, 2022.
- [4] A. Rajput, A. Dubey, R. Thakur, D. Singh, and U. P. Singh, "Career craft AI: A personalized resume analysis and job recommendations system," in *2024 1st International Conference on Innovative Sustainable Technologies for Energy, Mechatronics, and Smart Systems (ISTEMS)*, Dehradun, India, 2024.
- [5] S. Panchasara, R. K. Gupta, and A. Sharma, "AI based job recommendation system using BERT," in *2023 7th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*, Pune, India, 2023.
- [6] T. G. Sougandh, S. S. K. N. S. Reddy, and M. Belwal, "Automated resume parsing: A natural language processing approach," in *2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, Bangalore, India, 2023.
- [7] D. Abisha et al., "Respar: AI-driven resume parsing and recruitment system using NLP and generative AI," in *2024 Second International Conference on Intelligent Cyber Physical Systems and Internet of Things (ICoICI)*, Coimbatore, India, 2024.
- [8] A. P. A. K. K. S. Karthick Bharadwaj, and M. Venugopalan, "Semantic similarity analysis for resume filtering using PySpark," in *2024 IEEE 9th International Conference for Convergence in Technology (I2CT)*, Pune, India, 2024.
- [9] B. Bhardwaj, S. I. Ahmed, J. Jaiharie, R. Sorabh Dadhich, and M. Ganesan, "Web scraping using summarization and named entity recognition (NER)," in *2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS)*, Coimbatore, India, 2021.
- [10] A. R. Nair, S. M. and D. Gupta, "Comparative analysis of word embeddings for text classification in Spark NLP," in *2023 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM)*, Mysuru, India, 2023.
- [11] M. D. Amin, A. C. Harkare, N. S. Parmar, R. R. Wadhwa, and R. A. Patil, "Real time data based automated resume classification and job matching using SVC, Jaccard index and cosine similarity," in *2023 7th International Conference on Computer Applications in Electrical Engineering-Recent Advances (CERA)*, Roorkee, India, 2023.
- [12] V. Pendyala, N. Atrey, T. Aggarwal, and S. Goyal, "Artificial intelligence enabled, social media leveraging job matching system for employers and applicants," in *2022 International Conference on Recent Trends in Microelectronics, Automation, Computing and Communications Systems (ICMACC)*, Hyderabad, India, 2022.
- [13] A. Mukherjee and U. S. M., "Resume ranking and shortlisting with DistilBERT and XLM," in *2024 IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE)*, Bangalore, India, 2024.
- [14] X. Xue, et al., "Fine-grained semantics-enhanced graph neural network model for person-job fit," *Entropy*, vol. 27, no. 7, p. 703, 2025.
- [15] A. Patil, et al., "A survey on artificial intelligence (AI) based job recommendation systems," in *2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS)*, IEEE, 2023.
- [16] P. Singla and V. Verma, "A hybrid approach for job recommendation systems," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2024.
- [17] R. Dugyala, et al., "Smart recruitment system," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, 2024.
- [18] T. Ding, Y. Liu, and S. Wang, "Dual-branch mutual learning framework for job-resume matching," Available at SSRN 5108034.
- [19] Y. Li, et al., "Deep entity matching with pre-trained language models," arXiv preprint arXiv:2004.00584, 2020.
- [20] Dharmendra, A. Tripathi, K. Meenakshi, and D. Bhargava, "NLP-powered resume screening and ranking system," in *2025 3rd International Conference on Disruptive Technologies (ICDT)*, IEEE, 2025.
- [21] B. Gunaseelan, S. Mandal, and V. Rajagopalan, "Automatic extraction of segments from resumes using machine learning," in *2020 IEEE 17th India Council International Conference (INDICON)*, IEEE, 2020.
- [22] A. K. Sinha, M. A. K. Akhtar, and M. Kumar, "Automated resume parsing and job domain prediction using machine learning," *Indian Journal of Science and Technology*, vol. 16, no. 26, pp. 1967-1974, 2023.